



Laboratorio 2

Estructura de datos y algoritmos

Agustín González Incháustegui

Índice

- 1 Introducción**
- 2 Metodología e Implementación**
 - 2.1 Clase Voto
 - 2.2 Clase Candidato
 - 2.3 Clase Votante
 - 2.4 Clase UrnaElectoral
 - 2.5 Pruebas método Main
- 3 Análisis**
 - 3.1 Complejidad temporal
 - 3.2 Gestión de memoria
 - 3.3 Ventajas y desventajas
- 4 Conclusiones**

1. Introducción

En esta experiencia de laboratorio el problema inicial dice que la universidad necesita un sistema electrónico para las elecciones del centro de alumnos de la escuela de informática. La universidad quiere garantizar la integridad de los votos, que no existan duplicaciones en estos y que se puedan consultar los datos y resultados de la elección.

2. Metodología e Implementación

a) Clase voto: Esta clase representa el voto, como atributos tiene un identificador específico del voto, un identificador del/la estudiante que realizó el voto, y una marca de tiempo para saber a que hora se realizó. A continuación una foto de los atributos:

```
private int id; 3 usages
private int votanteID; 3 usages
private int candidatoID; 3 usages
private String timestamp; 3 usages
```

b) Clase Candidato: Esta clase representa al candidato que realiza el voto. La clase cuenta con los atributos: identificador para el candidato, nombre, partido, y una cola de objetos heredados de la clase Voto. A continuación una foto de la los atributos:

```
private int id; 3 usages
private String nombre; 3 usages
private String partido; 3 usages
private Queue<Voto> votosRecibidos; 4 usages
```

Además, esta clase cuenta con el metodo agregarvoto() utilizando la informacion de la clase voto para agregarlo a la cola.

```
public void agregarVoto(Voto voto) { no usages
    this.votosRecibidos.add(voto);
}
```

c) Clase Votante: Esta clase verifica/comprueba si una persona esta habilitada (o no) para votar, esto por si ya votó una vez o no. La clase tambien tiene atributos de identificador de voto y de persona (string). A continuación sus atributos:

```
private int id; 3 usages
private String nombre; 3 usages
private boolean yaVoto; 3 usages
```

La clase tambien cuenta con le metodo booleano marcarComoVotado() el cual verifica si votó o no

```
public void marcarComoVotado(){ no usages
    this.yaVoto = true;
}
```

2. Implementación

d) Clase Urna Electoral: Esta clase es la representacion electronica de la urna. Tiene como funciones registrar, reportar y ver los resultados de los votos. Sus atributos son una lista enlazada para los candidatos, una pila de votos, una fila de votos reportados y un contador (para los votos). A continuacion una foto de los atributos:

```
private LinkedList<Candidato> listaCandidatos; 4 usages
private Stack<Voto> historialVotos; 2 usages
private Queue<Voto> votosReportados; 2 usages
private int idCounter; 2 usages
```

La clase tambien cueneta con los siguientes metodos:

agregarCandidato() : el cual agrega un candidato habilitado a la lista.

registrarVoto() : el cual registra dentro de la pila el voto realizado.

reportarVoto() : el metodo busca el voto reportado y lo pasa a votosReportados.

obtenerResultados() : este metodo busca el nombre del candidato y lo mapea con la cantidad de votos que tienen. en otras palabras sirve para llevar la cuenta de los votos.

Finalmente ser creó un metodo main para realizar pruebas, la cuales se ven así:

```
Resultados (sin filtro)
Agustin: 3 votos
Nicolas: 1 votos
Resultados finales (filtrados)
Agustin: 2 votos
Nicolas: 1 votos
Agustin
Voto ID 2 a las 23:16:25
Voto ID 4 a las 23:16:25
Nicolas
Voto ID 3 a las 23:16:25
```

3. Análisis

3.1 Complejidad temporal

Metodo	Complejidad
registrarVoto	$O(k)$
obtenerResultados	$O(k)$
reportarVoto	$O(n + m)$

Table 1: tabla de complejidad de algunos metodos

3.2 Gestión de memoria

Para calcular el espacio requerido en este caso, se debe aplicar la siguiente formula:

Espacio total = Cantidad de votantes x 64 bytes

Espacio total = 10.000.000 x 64 bytes

Espacio total = 640.000.000 bytes

Como el resultado está en bytes se puede tambien hacer la siguiente relacion:

1024 **bytes** → 1 **kilobyte**

640.000.000 **bytes** → 625.000 **kilobytes**

1024 **kilobytes** → 1 **megabyte**

625.000 **kilobytes** → 610.35 **megabytes**

Entonces lo minimo que se necesita de espacio son 610.35 megabytes.

3.3 Ventajas y desventajas

Una ventaja en la implementacion de listas enlazadas es que los metodos para agregar y eliminar votos son de complejidad $O(1)$ lo que aumenta la eficiencia del algoritmo. Así mismo facilita el trabajo con las colas de votos, al cumplir con el orden FIFO.

Una desventaja es el uso de memoria, ya que los nodos guardan las referencias tanto del siguiente y del anterior nodo, aumentando el uso de memoria.

Para un sistema de votaciones en multiples facultades, se podria crear una nueva clase Facultad la cual debe tener sus propios candidatos, votos y votantes. Además, se debiese asignar a esta clase una nueva urna, así las votaciones serian independientes en cada facultad.

4. Conclusiones

En esta experiencia se implementó un sistema de votacion electronico mediante cuatro clases principales, ademas de un método main principal. Las clases representan los votos, la identificacion del voto, del votante y del candidato, de esta manera se guardan los respectivos datos. Inicialmente para resolver o solicitado se pensó en como modular cada elemento que representa una votacion, como se menciona anteriormente (votos, votante, caniddatos, etc). Se decidió que para la clase Candidato se incluyera un cola para almecenar la cuenta de cada voto. Para la clase UrnaElectoral se implementa una pila para el historial de votacion. El metodo main para probar el codigo. Un obstaculo en la implementacion fue el del metodo reportarVoto() ya que el método está compuesto por una cola y esta debe ser recorrida completa siempre. No obstante se logró la implementacion correcta.

5. Referencias

Link de github: <https://github.com/agustin357/Lab-2.git>